

Breakable objects system: Simple solution



Main features

1. Procedural mesh fracturing right in Unity Editor. Textured meshes are supported.
2. A lot of options for broken objects
3. Shards pools that can be used to share the same shards between similar objects

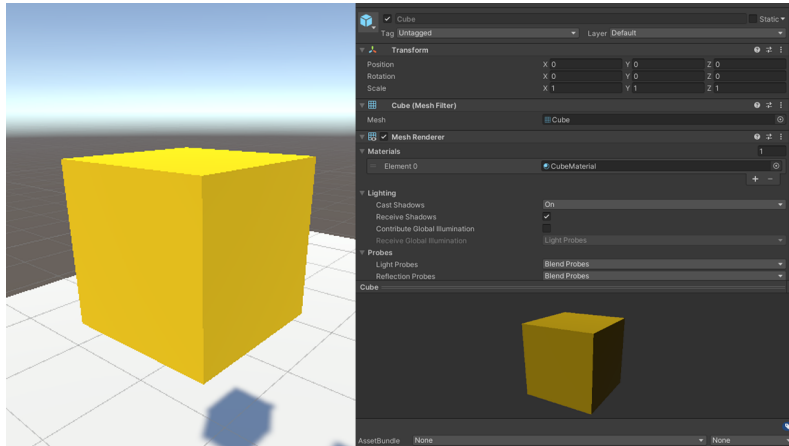
Important notes

Please note that the provided solution is SIMPLE. So, there are some points

1. Destruction algorithm works by slicing with random planes
2. By default the new geometry will be generated for inner surfaces. Interpolated UV-coordinates will be applied. It will work properly only if your mesh is convex. If your mesh is concave or too difficult you may encounter some problems or graphical artifacts. You can play around with some values or disable option “**Add new geometry**” in this case

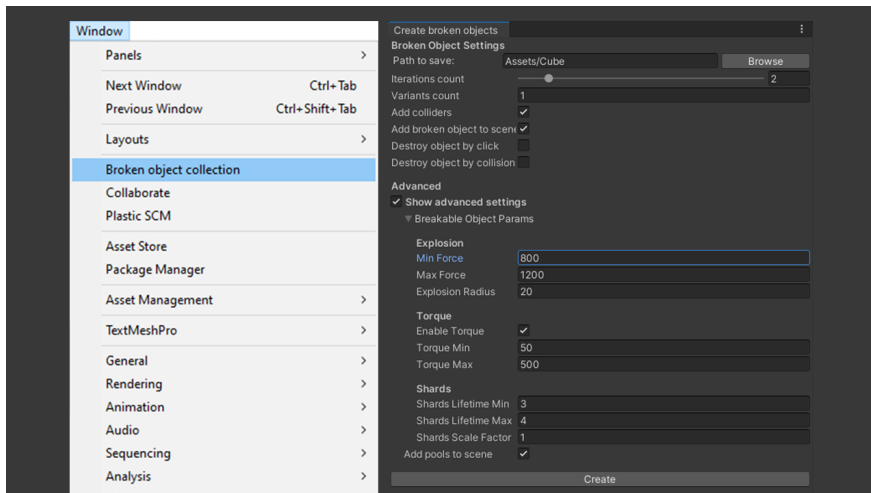
Quick start

Select game object with mesh filter



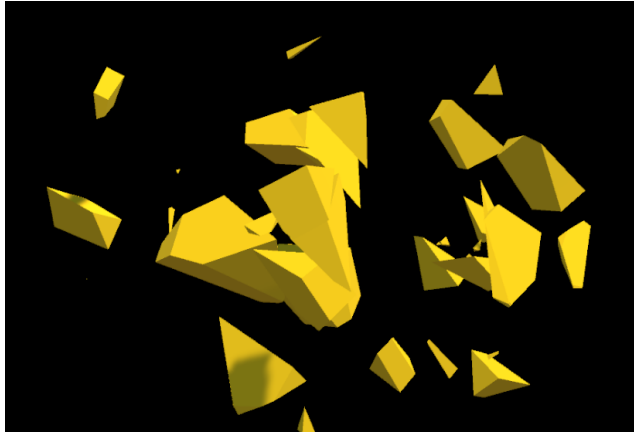
Select any game object which has **Mesh Filter Component**

Open broken "Create broken objects" window



1. Go to **Window -> Create broken objects**
2. Select game object which has **Mesh Filter Component**
3. Check toggles:
 1. **Add colliders to shards**
 2. **Add broken object to scene**
 3. **Destroy object by click**
4. Press **Create**

Run and destroy



1. Run scene
2. Click on object to destroy it

"Create broken objects" window

Create broken objects

Broken Object Settings

Path to save: Assets/Cube

Browse

Iterations count

2

Variants count

1

Add colliders to shards ☒

Shard collider type

Default

Add broken object to scene ☒

Add new geometry (Works) ☒

Destroy object by click ☐

Destroy object by collision ☐

Advanced

☒ Show advanced settings

Breakable Object Params

Explosion

Min Force

100

Max Force

200

Explosion Radius

1

Torque

Enable Torque ☒

Torque Min

50

Torque Max

500

Shards

Shards Lifetime Min

3

Shards Lifetime Max

4

Shards Scale Factor

1

Add pools to scene ☒

Create

Path to save: select path where you want to save broken objects. Only the path which is inside the "Assets" folder is valid.

Iterations count: More iterations - more shards. 10 iterations ~ 1000 shards

Add colliders to shards: Should shards collide with each other and with objects on the scene?

Shard collider type:

Mesh collider will be applied to each shard by default. Use other types of colliders if you want some optimization

Add broken object to scene: Adds broken version of object to scene near original object

Add new geometry:

Generate surfaces inside broken objects. UV coordinates will be taken from near vertices and interpolated. IMPORTANT: It works properly only if your mesh is convex. If your mesh is concave or too difficult you may encounter some problems or graphical artifacts.

Destroy object by click:

Adds special component which will react to mouse click or finger tap and destroy object

Destroy object by collision:

Adds special component which will react to collisions and destroy object

Object collider type:

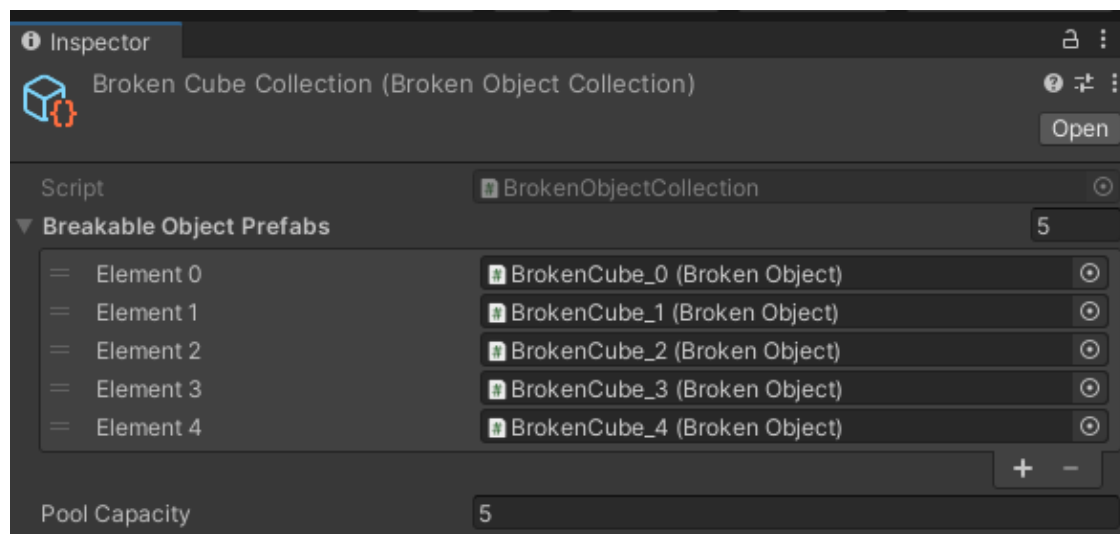
Collider should be applied to the main object to detect clicks or collisions. Mesh collider will be applied by default. Use other types of colliders if you want some optimization

Advanced options

These options also can be found at the created **BreakableObject** component. They determine the flight speed and the lifetime of the shards.

Broken object collections

Broken object collection is a scriptable object which contains all variants of broken object



Broken objects will be retrieved from the collection randomly at the runtime.

Here you can also set an initial pool capacity **for each** variant. It means that on first use (or pool initialization if you call a **BrokenObjectPools.AddCollectionToPool**) several instances of each broken variant will be added to a pool.

Pools are expandable. Therefore, if you break a lot of objects at runtime the new broken objects variations will be instantiated and added to pools (see next page about the pools)

Broken object collection will be created automatically after pressing the **Create** button in the **Create broken objects** window.

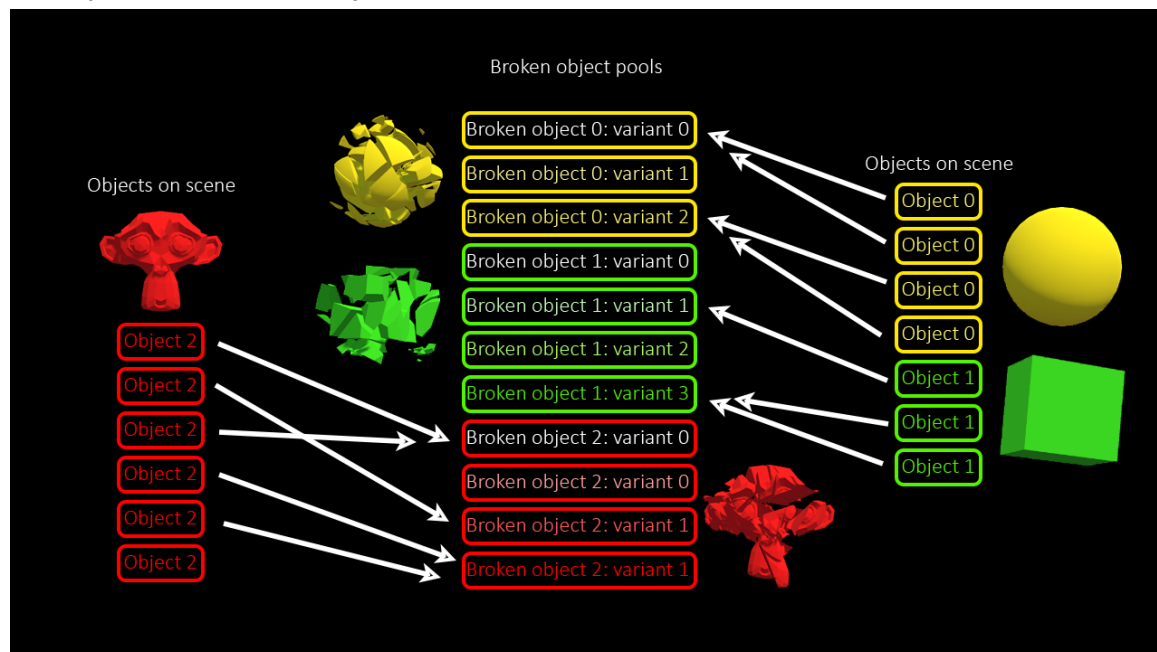
You can also create a **BrokenObjectCollection** yourself by following:**Assets - Create - Broken object collection - Empty**

Broken object pools

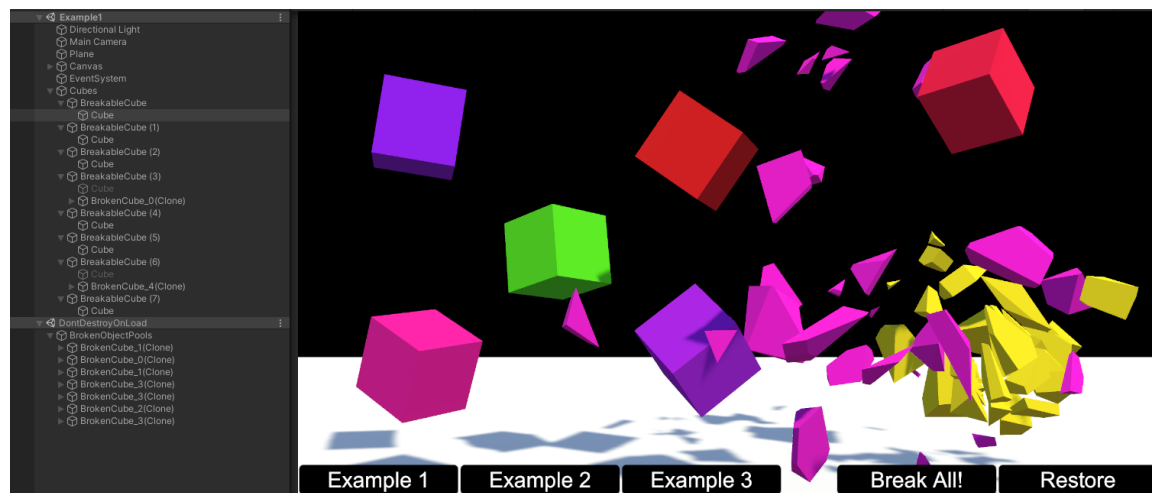
Why do I need that?

Pool is a collection of objects that are kept ready to use. You can retrieve objects from the pool and then release them after using them. Main goal of it is to avoid memory allocations and slow Game Object instantiations.

The Breakable Objects system will automatically place shards to the **Broken Object Pools** during the destruction process. It means that shards from one broken object will be reused on another identical object later. It greatly increases runtime performance especially if you have to destroy a lot of identical objects.



All you need to do is just to ensure that the **BrokenObjectPools** game object exists on your scene. Please do not delete it!



Code

You can find examples of using breakable objects in the **Examples** folder

BreakableObject

Public methods

public void Break(Vector3 explosionPosition = default)

Breaks object. This method initiates an explosion inside an object, replaces original mesh with broken version and applies a force to the shards

Vector3 explosionPosition: You can set a custom explosion position. Otherwise center of the object will be used

public void Restore()

Instantly restores object

Properties

public GameObject OriginalObject

Instance of the original object

public BreakableObjectParams Params

Parameters of the explosion: force, shards lifetime, etc.

public BrokenObjectCollection BrokenObjectsCollection

A **BrokenObjectCollection** instance from which broken objects will be retrieved

public bool Broken { get; private set; }

Is object currently in broken state. It becomes true right after **Break** was called

public bool IsBreakingInProgress { get; private set; }

True until the last shard is gone

Break by click

Add this component to the breakable object to destroy object by click or tap

Break by collision

Add this component to the breakable object to destroy object by any collision

OnBreakBehaviourAbstract

You can derive from this class to create an additional break behavior. See **example 2**

Public methods

```
public abstract void OnBreak();
```

OnRestoreBehaviourAbstract

You can derive from this class to create an additional restore behavior

Public methods

```
public abstract void OnRestore();
```

BrokenObjectPools

Public methods

```
public void AddCollectionToPool(BrokenObjectCollection brokenObjectCollection)
```

You can pre-init a pool with a broken collection. This way you'll save a performance later. See **example 1**

```
public BrokenObject GetRandomObjectFromCollection(BrokenObjectCollection  
collection, Transform parent, Vector3 position, Quaternion rotation, Vector3 localScale,  
out int index)
```

Retrieves random broken object variant from the target **BrokenObjectCollection** at the target transform.

```
public void ReleaseObjectFromCollection(BrokenObjectCollection collection,  
BrokenObject go, int index)
```

Releases the broken object to the collection under the target index

Roadmap

1. New algorithms for procedural mesh destruction
2. Beautiful animated object restoration from shards
3. Baking destruction to animation
4. Further optimization, more smart pools
5. More settings for better and flexible destruction process configuration
6. More examples